

Informe de laboratorio 5: Algoritmos de Búsqueda con BFS, DFS, UCS y A*

1. Decisiones de diseño

1.1. Una sola interface para los dos escenarios

Lo primero fue no escribir cada algoritmo dos veces. En lugar de tener un BFS para el grafo y otro para el laberinto, hacer que los algoritmos sean **agnósticos al problema**: solo le preguntan cuatro cosas a una interface común (**Problema**):

Pregunta	Método	En el grafo	En el laberinto
¿Dónde empiezo?	<code>estado_inicial()</code>	nodo "A"	celda (0,0)
¿Llegué?	<code>es_objetivo(e)</code>	<code>e == "E"</code>	<code>e == (9,14)</code>
¿A dónde puedo ir y a qué costo?	<code>vecinos(e)</code>	aristas del nodo	celdas vecinas (costo 1)
¿Cuánto me falta (estimado)?	<code>heuristica(e)</code>	distancia en línea recta	distancia Manhattan

ProblemaGrafo y **ProblemaLaberinto** implementan esa interface, así que los 4 algoritmos los escribí una sola vez y sirven para ambos escenarios. Lo único que cambia es cómo se generan los vecinos (en el grafo son las aristas; en el laberinto, las celdas de al lado). Aparte de ahorrar código, me gusta porque deja claro que la búsqueda es la misma idea sin importar el escenario o problema, que es justamente lo que buscan resolver problemas matemáticos.

1.2. Un solo esqueleto, distinta "frontera"

Los cuatro algoritmos comparten la misma estructura de búsqueda en grafo:

- **frontera**: los nodos que ya descubrí pero todavía no expandí (los pendientes).
- **explorados**: los que ya procesé (para no repetir trabajo ni quedarme en ciclos).
- **padre**: de quién vengo a cada nodo, para reconstruir el camino al final yendo hacia atrás (así no tengo que guardar el camino entero en cada nodo, que sería un gasto de memoria innecesario).

Lo único que cambia de un algoritmo a otro es de qué estructura sacar el siguiente nodo:

Algoritmo	Frontera	Sacar primero
BFS	Cola FIFO (<code>deque</code>)	el más antiguo
DFS	Pila LIFO (<code>list</code>)	el más reciente
UCS	Cola de prioridad (<code>heapq</code>)	el de menor $g(n)$
A*	Cola de prioridad (<code>heapq</code>)	el de menor $f(n)=g(n)+h(n)$

UCS y Alos junté en una sola función a la que le paso cómo calcular la prioridad: UCS usa g , A usa $g + h$. Creo que es la forma más directa de mostrar que A* es literalmente UCS + heurística (cuando $h=0$, A* se convierte en UCS).

1.3. Detallitos

- **Desempate estable en la cola de prioridad:** al `heapq` le agrego un contador incremental como segundo criterio. Si dos nodos tienen la misma prioridad, gana el que entró primero. Esto hace el recorrido reproducible y además evita que Python intente comparar los estados entre sí (tiraría error con las tuplas).
- **Relajación (UCS/A*):** solo vuelvo a meter un vecino al heap si encontré un camino más barato hacia él (`nuevo_g < mejor_g[vecino]`). Las entradas viejas que quedan en el heap las descarto al sacarlas (`if estado in explorados: continue`).
- **Cuándo marco un nodo:** BFS lo marca al encolar (cada nodo entra una sola vez); DFS, UCS y A* lo marcan al expandir. La diferencia es a propósito, es la forma estándar y correcta de cada uno.
- **Heurísticas:**
 - Laberinto: **Manhattan** $|\Delta fila| + |\Delta col|$. Es admisible porque me muevo en 4 direcciones con costo 1, así que nunca puede sobreestimar el costo real.
 - Grafo: **distancia euclidiana** a las coordenadas del objetivo. Es admisible cuando los costos de las aristas son $\geq$ la distancia en línea recta. Si el grafo no trae coordenadas, `heuristica()` devuelve 0 y A* se comporta como UCS.

1.4. Visualización

- **Laberinto:** `matplotlib.imshow` con colores discretos (barrera / libre / visitado / camino / inicio / objetivo).
- **Grafo:** `networkx` para dibujar; el camino final queda resaltado con aristas gruesas naranjas y los nodos pintados según su rol.
- **Respaldo en texto:** si matplotlib no está instalado, todo se muestra en consola con símbolos, así el programa nunca se queda sin mostrar nada.

2. Resultados

2.1. Grafo (inicio = A, objetivo = E)

Algoritmo	Camino	Long. (nodos)	Costo	Visitados
BFS	A → B → D → E	4	12	6
DFS	A → B → C → D → E	5	14	5
UCS	A → B → C → F → E	5	10	6
A*	A → B → C → F → E	5	10	5

Este grafo lo armé a propósito para que BFS y UCS dieran distinto:

- **BFS no es óptimo en costo:** agarra el camino con menos aristas (3 saltos), pero ese camino cuesta 12, más que el óptimo.
- **DFS es el peor en calidad:** costo 14, se hunde por una rama sin mirar el costo.
- **UCS sí es óptimo en costo** (10), aunque su camino tenga más saltos (4).
- **A* llega al mismo óptimo (10) visitando menos nodos** (5 contra 6): la heurística lo guía hacia E y se ahorra expandir nodos innecesariamente.

2.2. Laberinto (inicio = (0,0), objetivo = (9,14))

Algoritmo	Long. camino	Costo	Visitados
BFS	30	29	78
DFS	32	31	89
UCS	30	29	78
A*	30	29	54

Acá todos los pasos cuestan 1, así que:

- **BFS, UCS y A*** dan el mismo costo óptimo (29). BFS y UCS incluso expanden los mismos 78 nodos (con costo uniforme, UCS es BFS).
- **A*** es el más eficiente de lejos: mismo óptimo (29) pero explorando solo **54 nodos** (como un 30% menos), porque Manhattan lo empuja derecho hacia el objetivo.
- **DFS vuelve a ser el peor**: camino más largo (32 pasos) y encima visita *más* nodos (89), porque deambula en profundidad sin rumbo a la meta.

3. Comparación general

Criterio	BFS	DFS	UCS	A*
Estructura de frontera	Cola FIFO	Pila LIFO	Cola prioridad (g)	Cola prioridad (g+h)
¿Encuentra solución?	Sí (si existe)	Sí (si existe)	Sí	Sí
¿Óptimo?	Solo si costos iguales	No	Sí (siempre)	Sí (si h admisible)
¿Usa el costo?	No	No	Sí	Sí
¿Usa heurística?	No	No	No	Sí
Nodos explorados	Muchos	Variable	Muchos	Pocos
Memoria	Alta	Baja	Alta	Alta

Conclusión. BFS y DFS son no informados: ignoran el costo. BFS me garantiza el camino con menos pasos (óptimo solo si todos los costos son iguales) pero a costa de mucha memoria; DFS gasta poca memoria pero no me asegura nada sobre la calidad. UCS ya considera el costo real y garantiza el óptimo, pero explora a ciegas en todas direcciones. **A* se queda con lo mejor de ambos mundos:** garantiza el óptimo (con heurística admisible) y, al guiarse por la misma heurística $h(n)$, explora muchos menos nodos. En problemas donde tengo una buena heurística (como Manhattan en el laberinto) A* es la mejor opción de lejos.